

```

In [ ]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bits")
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.")
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

```

```

# Fire
vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f"  S2MM:{hex(s2mm)}  MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

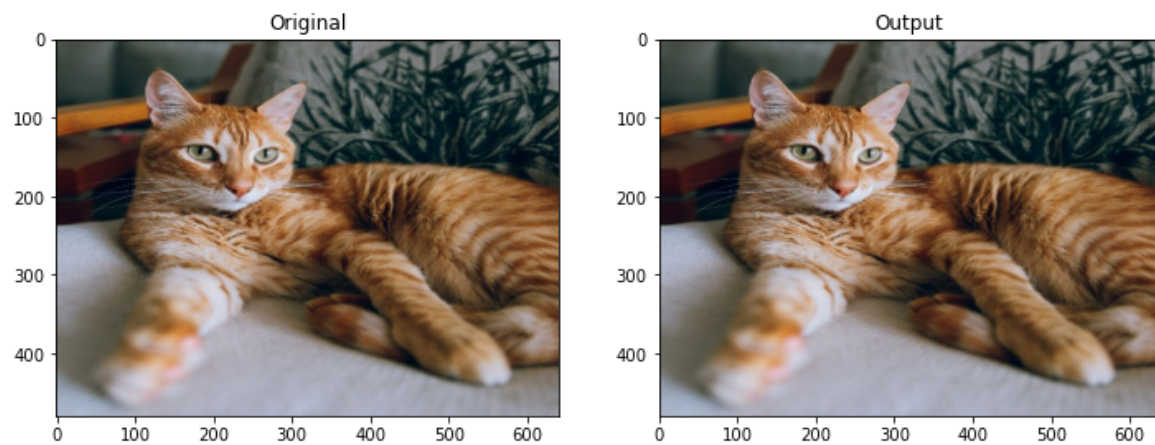
in_buffer.freebuffer()
out_buffer.freebuffer()

```

```

Loaded OK
Image loaded: True
in_buffer:  0x16900000
out_buffer: 0x16b00000
VDMA base: 0x43000000
After reset MM2S:0x10082 S2MM:0x10082
After run  MM2S:0x10000 S2MM:0x10000
Before fire MM2S:0x10000 S2MM:0x10000
Polling...
  S2MM:0x14100 MM2S:0x10000
  S2MM:0x15100 MM2S:0x11000
Done
Unique: [0 1 2 3 4 5 6 7 8 9]

```



```

In [ ]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time
overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bits")
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.")
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Fire
vdma.write(0xA0, IMG_HEIGHT)

```

```
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

```

In [ ]: import numpy as np
import cv2
import matplotlib.pyplot as plt
import time
overlay = Overlay("/home/xilinxfrom pyng import Overlay, MMIO, allocate
/jupyter_notebooks/image_enhancer/Bitstreamfiles/gray_test.bit")
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.")
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Fire
vdma.write(0xA0, IMG_HEIGHT)

```

```

vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f"  S2MM:{hex(s2mm)}  MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()

```

```

Loaded OK
Image loaded: True
in_buffer: 0x16900000
out_buffer: 0x16b00000
VDMA base: 0x43000000
After reset MM2S:0x10082 S2MM:0x10082
After run MM2S:0x10000 S2MM:0x10000
Before fire MM2S:0x10000 S2MM:0x10000
Polling...
  S2MM:0x10000 MM2S:0x10000
  S2MM:0x11000 MM2S:0x11000
Done
Unique: [ 1  2  3  4  5  6  7  8  9 10]

```

In []:

```

In [1]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time
overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Fire
vdma.write(0xA0, IMG_HEIGHT)

```



```
vdma.write(0x50, IMG_HEIGHT)

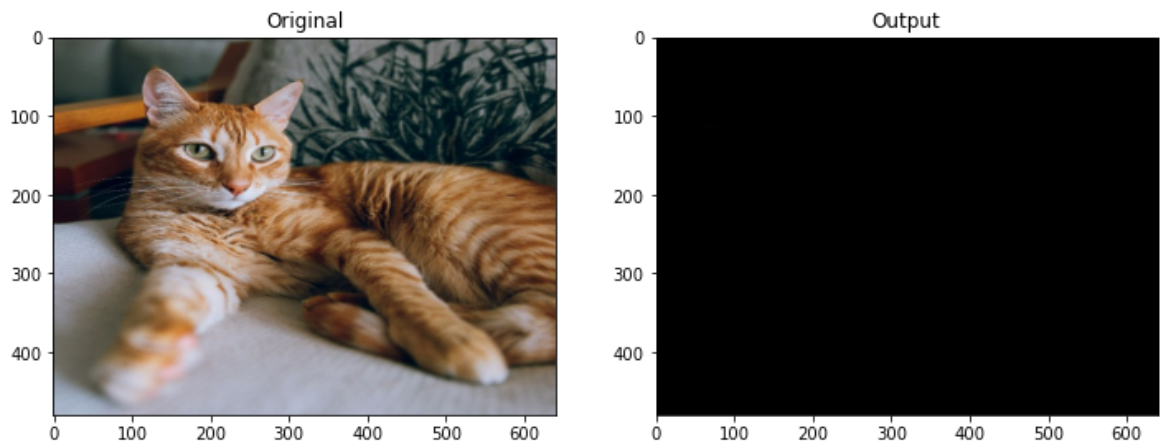
print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

```
Timeout
Unique: [ 0 11 12 13 16 20 26 28 255]
```



```

In [1]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer:  {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Fire

```

```
vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

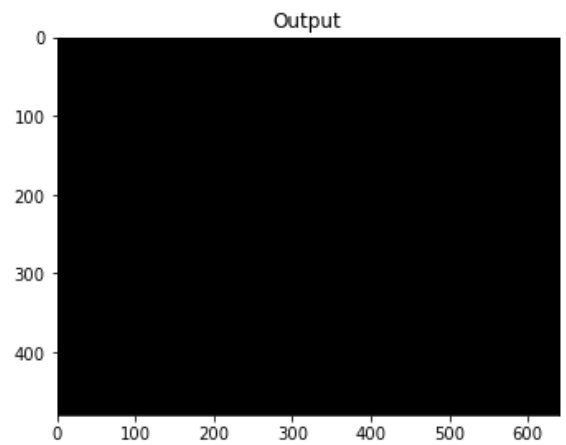
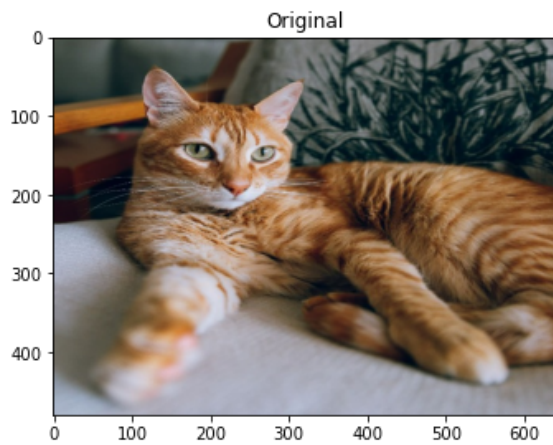
print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

```
Timeout
Unique: [0]
```



```

In [2]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded OK")

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)
print(f"VDMA base: {hex(overlay.ip_dict['axi_vdma_0']['phys_addr'])}")

# Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

# Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# Hsize stride
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

# Fire

```

```
vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

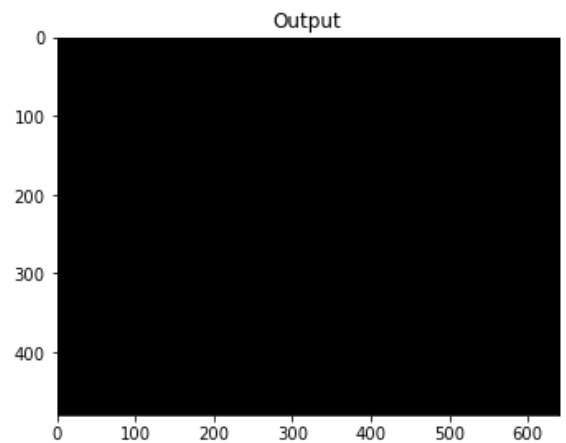
print("Polling...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
    if s2mm & 0x1000:
        print("Done")
        break
    time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

```
Timeout
Unique: [0]
```




```

In [1]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE      = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer:  {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.
```

Input In [1]

```
time.sleep(0.
```

^

SyntaxError: incomplete input

```

In [2]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer:  {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.
```

Input In [2]

```
time.sleep(0.
```

^

SyntaxError: incomplete input

```

In [3]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer:  {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f"  S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("FPGA Gray Output")
plt.show()

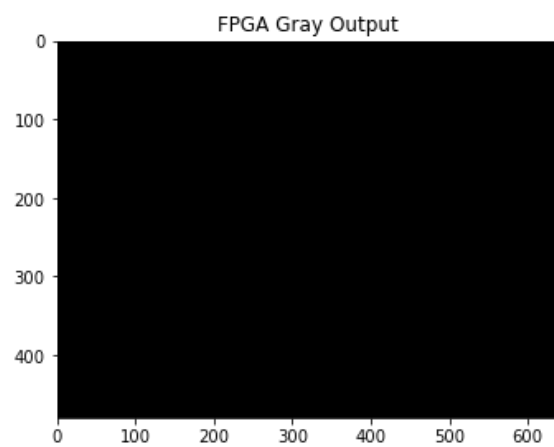
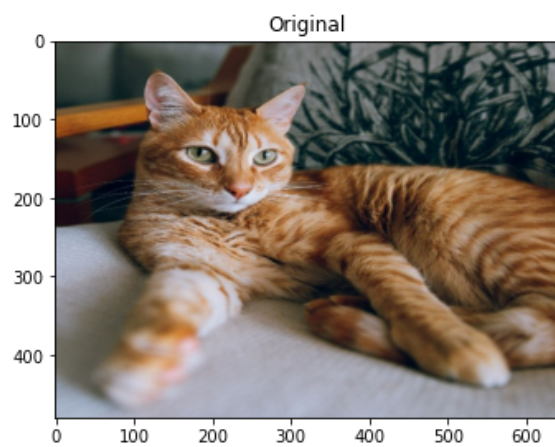
in_buffer.freebuffer()
out_buffer.freebuffer()
```

[illegible]

```
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
S2MM:0x10000 MM2S:0x10000
```

Timeout

Unique: [0]




```

In [ ]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitsp
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f"  S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("FPGA Gray Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

```

In [1]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitstream")
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.jpg")
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer: {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("FPGA Gray Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

Input In [1]

```
rom pyng import Overlay, MMIO, allocate
      ^
```

SyntaxError: invalid syntax

```

In [2]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

overlay = Overlay("/home/xilinx/jupyter_notebooks/image_enhancer/Bitstream")
print("Loaded:", list(overlay.ip_dict.keys()))

IMG_WIDTH  = 640
IMG_HEIGHT = 480
STRIDE     = IMG_WIDTH * 4

img = cv2.imread("/home/xilinx/jupyter_notebooks/image_enhancer/test.jpg")
print(f"Image loaded: {img is not None}")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

print(f"in_buffer:  {hex(in_buffer.physical_address)}")
print(f"out_buffer: {hex(out_buffer.physical_address)}")

vdma = MMIO(overlay.ip_dict['axi_vdma_0']['phys_addr'], 0x10000)

vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.5)
print(f"After reset MM2S:{hex(vdma.read(0x00))} S2MM:{hex(vdma.read(0x30))}")

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)
print(f"After run MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print(f"Before fire MM2S:{hex(vdma.read(0x04))} S2MM:{hex(vdma.read(0x34))}")

vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling...")
timeout = time.time() + 15.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)

```

```
mm2s = vdma.read(0x04)
print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")
if s2mm & 0x1000:
    print("Done")
    break
time.sleep(0.2)
else:
    print("Timeout")

out_buffer.invalidate()
out_array = np.array(out_buffer)
print(f"Unique: {np.unique(out_array)[:10]}")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(out_array, cv2.COLOR_RGBA2RGB))
plt.title("FPGA Gray Output")
plt.show()

in_buffer.freebuffer()
out_buffer.freebuffer()
```

Input In [2]

```
rom pyng import Overlay, MMIO, allocate
      ^
```

SyntaxError: invalid syntax

```

In [3]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

# 1. Load Bitstream
overlay = Overlay("./Bitstreamfiles/loop_again_main_wrapper.bit")
vdma_base = overlay.ip_dict['axi_vdma_0']['phys_addr']
vdma = MMIO(vdma_base, 0x10000)

# 2. Setup Dimensions
W, H = 640, 480
STRIDE = W * 4

# 3. Clean Memory Allocation
in_buf = allocate(shape=(H, W, 4), dtype=np.uint8)
out_buf = allocate(shape=(H, W, 4), dtype=np.uint8)

# Create a very bright test pattern (Solid Red) to make it obvious if
in_buf[:] = [255, 0, 0, 255]
out_buf[:] = 0
in_buf.flush()
out_buf.flush()

# 4. HARD RESET VDMA
vdma.write(0x00, 0x4) # MM2S Reset
vdma.write(0x30, 0x4) # S2MM Reset
time.sleep(0.5)

# 5. Start and Configure
vdma.write(0x00, 0x1) # MM2S Run
vdma.write(0x30, 0x1) # S2MM Run
vdma.write(0x5C, in_buf.physical_address)
vdma.write(0xAC, out_buf.physical_address)
vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

# FIRE
vdma.write(0xA0, H)
vdma.write(0x50, H)

print("Polling...")
for _ in range(10):
    s2mm_stat = vdma.read(0x34)
    mm2s_stat = vdma.read(0x04)

    # Check for Errors (Bits 4, 5, 6 of Status Registers)
    if s2mm_stat & 0x4000: print("!!! S2MM Internal Error !!!")
    if s2mm_stat & 0x2000: print("!!! S2MM Slave Error !!!")

    if s2mm_stat & 0x1000:
        print("SUCCESS: Loopback Complete.")

```

```
        break
    time.sleep(0.5)
else:
    print(f"FAILED: S2MM={hex(s2mm_stat)} MM2S={hex(mm2s_stat)}")

out_buf.invalidate()
# Check if any pixels changed from 0
if np.any(out_buf):
    print("Data found in output buffer!")
else:
    print("Output buffer is still all zeros.")

in_buf.freebuffer()
out_buf.freebuffer()
```



```

-----
-----
RuntimeError                                Traceback (most recent call
last)
Input In [3], in <cell line: 8>()
      5 import time
      7 # 1. Load Bitstream
----> 8 overlay = Overlay("./Bitstreamfiles/loop_again_main_wrapper.b
it")
      9 vdma_base = overlay.ip_dict['axi_vdma_0']['phys_addr']
     10 vdma = MMIO(vdma_base, 0x10000)

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/ove
rplay.py:319, in Overlay.__init__(self, bitfile_name, dtbo, download,
ignore_version, device, gen_cache)
     315 super().__init__(bitfile_name, dtbo, partial=False, device=de
vice)
     317 self._register_drivers()
--> 319 self.device.set_bitfile_name(self.bitfile_name)
     320 self.parser = self.device.parser
     322 self.ip_dict = (
     323     self.gpio_dict
     324 ) = (
     325     self.interrupt_controllers
     326 ) = self.interrupt_pins = self.hierarchy_dict = dict()

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/device.py:118, in Device.set_bitfile_name(self, bitfile_name)
     116 def set_bitfile_name(self, bitfile_name: str) -> None:
     117     self.bitfile_name = bitfile_name
--> 118     self.parser = self.get_bitfile_metadata(self.bitfile_nam
e)
     119     self.mem_dict = self.parser.mem_dict
     120     self.ip_dict = self.parser.ip_dict

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/embedded_device.py:692, in EmbeddedDevice.get_bitfile_metadata
(self, bitfile_name, partial)
     690 parser = _get_bitstream_handler(bitfile_name).get_parser(part
ial=partial)
     691 if parser is None:
--> 692     raise RuntimeError("Unable to find metadata for bitstrea
m")
     693 return parser

RuntimeError: Unable to find metadata for bitstream

```

```
In [1]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time

print("Loading Fresh Bitstream...")
# Using the exact absolute path from your terminal
bitstream_path = "/home/xilinx/jupyter_notebooks/image_enhancer/Bitst
overlay = Overlay(bitstream_path)
print("Loaded IPs:", list(overlay.ip_dict.keys()))

IMG_WIDTH = 640
IMG_HEIGHT = 480
STRIDE = IMG_WIDTH * 4

# Load and prep image
print("Loading test.jpg...")
img = cv2.imread("test.jpg")
if img is None:
    raise ValueError("Could not find test.jpg! Make sure it is in the ")
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

# Allocate memory buffers
print("Allocating memory...")
in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)

# Write image to memory
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

# Setup VDMA MMIO
vdma_base = overlay.ip_dict['axi_vdma_0']['phys_addr']
vdma = MMIO(vdma_base, 0x10000)

print("Initializing VDMA...")
# 1. Reset
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.2)

# 2. Run
vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)

# 3. Addresses
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

# 4. Hsize and Stride
vdma.write(0x54, STRIDE)
```

```
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

# 5. Fire (VSIZE)
print("Firing Transfer...")
vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

# 6. Polling
print("Polling VDMA Status...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")

    # Bit 12 (0x1000) is Interrupt on Complete / Bit 1 (0x2) is Idle
    if s2mm & 0x1000:
        print("Done! VDMA signaled completion.")
        break
    time.sleep(0.2)
else:
    print("Polling loop finished. Checking output buffer anyway...")

# 7. Invalidate and check data
out_buffer.invalidate()
unique_vals = np.unique(np.array(out_buffer))
print(f"Unique pixel values in output: {unique_vals[:10]}")

if len(unique_vals) > 1:
    print("SUCCESS: Data successfully moved through the loopback!")
else:
    print("FAILURE: Output buffer is still empty.")

# 8. Display
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(np.array(out_buffer), cv2.COLOR_RGBA2RGB))
plt.title("Loopback Output")
plt.show()

# 9. Clean up
in_buffer.freebuffer()
out_buffer.freebuffer()
```

Loading Fresh Bitstream...

```

-----
-----
RuntimeError                                Traceback (most recent call
last)
Input In [1], in <cell line: 10>()
      8 # Using the exact absolute path from your terminal
      9 bitstream_path = "/home/xilinx/jupyter_notebooks/image_enhan
cer/Bitstreamfiles/loop_again_main_wrapper.bit"
--> 10 overlay = Overlay(bitstream_path)
      11 print("Loaded IPs:", list(overlay.ip_dict.keys()))
      13 IMG_WIDTH = 640

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/ove
rlay.py:319, in Overlay.__init__(self, bitfile_name, dtbo, download,
ignore_version, device, gen_cache)
      315 super().__init__(bitfile_name, dtbo, partial=False, device=de
vice)
      317 self._register_drivers()
--> 319 self.device.set_bitfile_name(self.bitfile_name)
      320 self.parser = self.device.parser
      322 self.ip_dict = (
      323     self.gpio_dict
      324 ) = (
      325     self.interrupt_controllers
      326 ) = self.interrupt_pins = self.hierarchy_dict = dict()

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/device.py:118, in Device.set_bitfile_name(self, bitfile_name)
      116 def set_bitfile_name(self, bitfile_name: str) -> None:
      117     self.bitfile_name = bitfile_name
--> 118     self.parser = self.get_bitfile_metadata(self.bitfile_nam
e)
      119     self.mem_dict = self.parser.mem_dict
      120     self.ip_dict = self.parser.ip_dict

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/embedded_device.py:692, in EmbeddedDevice.get_bitfile_metadata
(self, bitfile_name, partial)
      690 parser = _get_bitstream_handler(bitfile_name).get_parser(part
ial=partial)
      691 if parser is None:
--> 692     raise RuntimeError("Unable to find metadata for bitstrea
m")
      693 return parser

RuntimeError: Unable to find metadata for bitstream

```

```
In [2]: import os
        from pynq import Overlay

        # 1. Paths
        folder = "/home/xilinx/jupyter_notebooks/image_enhancer/Bitstreamfiles"
        old_bit = folder + "loop_again_main_wrapper.bit"
        old_hwh = folder + "loop_again_main.hwh"
        new_bit = folder + "loop.bit"
        new_hwh = folder + "loop.hwh"

        # 2. Rename files to match (if they exist)
        if os.path.exists(old_bit):
            os.rename(old_bit, new_bit)
        if os.path.exists(old_hwh):
            os.rename(old_hwh, new_hwh)

        print("Files renamed to loop.bit and loop.hwh")

        # 3. Load the overlay again
        overlay = Overlay(new_bit)
        print("Loaded successfully!")
        print("IPs found:", list(overlay.ip_dict.keys()))
```

Files renamed to loop.bit and loop.hwh

```

-----
-----
RuntimeError                                Traceback (most recent call
last)
Input In [2], in <cell line: 20>()
      17 print("Files renamed to loop.bit and loop.hwh")
      19 # 3. Load the overlay again
--> 20 overlay = Overlay(new_bit)
      21 print("Loaded successfully!")
      22 print("IPs found:", list(overlay.ip_dict.keys()))

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/ove
rlay.py:319, in Overlay.__init__(self, bitfile_name, dtbo, download,
ignore_version, device, gen_cache)
      315 super().__init__(bitfile_name, dtbo, partial=False, device=de
vice)
      317 self._register_drivers()
--> 319 self.device.set_bitfile_name(self.bitfile_name)
      320 self.parser = self.device.parser
      322 self.ip_dict = (
      323     self.gpio_dict
      324 ) = (
      325     self.interrupt_controllers
      326 ) = self.interrupt_pins = self.hierarchy_dict = dict()

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/device.py:118, in Device.set_bitfile_name(self, bitfile_name)
      116 def set_bitfile_name(self, bitfile_name: str) -> None:
      117     self.bitfile_name = bitfile_name
--> 118     self.parser = self.get_bitfile_metadata(self.bitfile_nam
e)
      119     self.mem_dict = self.parser.mem_dict
      120     self.ip_dict = self.parser.ip_dict

File /usr/local/share/pynq-venv/lib/python3.10/site-packages/pynq/pl_
server/embedded_device.py:692, in EmbeddedDevice.get_bitfile_metadata
(self, bitfile_name, partial)
      690 parser = _get_bitstream_handler(bitfile_name).get_parser(part
ial=partial)
      691 if parser is None:
--> 692     raise RuntimeError("Unable to find metadata for bitstrea
m")
      693 return parser

RuntimeError: Unable to find metadata for bitstream

```

```

In [3]: import os
        from pynq import Overlay, MMIO, allocate
        import numpy as np
        import cv2
        import matplotlib.pyplot as plt
        import time

        folder = "/home/xilinx/jupyter_notebooks/image_enhancer/Bitstreamfiles

# 1. FOOLPROOF RENAME: Find any .hwh file and force it to match loop.b
print("Fixing filenames...")
for f in os.listdir(folder):
    if f.endswith(".hwh") and f != "loop.hwh":
        os.rename(os.path.join(folder, f), os.path.join(folder, "loop.hwh"))

print("Current files in folder:")
print([f for f in os.listdir(folder) if "loop" in f])

# 2. LOAD OVERLAY
print("\nLoading Fresh Bitstream...")
bitstream_path = folder + "loop.bit"
overlay = Overlay(bitstream_path)
print("Loaded IPs:", list(overlay.ip_dict.keys()))

# 3. SETUP VDMA TEST
IMG_WIDTH = 640
IMG_HEIGHT = 480
STRIDE = IMG_WIDTH * 4

print("Loading test.jpg...")
img = cv2.imread("test.jpg")
if img is None:
    raise ValueError("Could not find test.jpg! Make sure it is in the s
img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

print("Allocating memory...")
in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
in_buffer[:] = img_rgba
out_buffer[:] = 0
in_buffer.flush()
out_buffer.flush()

vdma_base = overlay.ip_dict['axi_vdma_0']['phys_addr']
vdma = MMIO(vdma_base, 0x10000)

print("Initializing VDMA...")
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.2)

vdma.write(0x00, 0x00000001)
vdma.write(0x30, 0x00000001)

```

```
vdma.write(0x5C, in_buffer.physical_address)
vdma.write(0xAC, out_buffer.physical_address)

vdma.write(0x54, STRIDE)
vdma.write(0x58, STRIDE)
vdma.write(0xA4, STRIDE)
vdma.write(0xA8, STRIDE)

print("Firing Transfer...")
vdma.write(0xA0, IMG_HEIGHT)
vdma.write(0x50, IMG_HEIGHT)

print("Polling VDMA Status...")
timeout = time.time() + 5.0
while time.time() < timeout:
    s2mm = vdma.read(0x34)
    mm2s = vdma.read(0x04)
    print(f" S2MM:{hex(s2mm)} MM2S:{hex(mm2s)}")

    if s2mm & 0x1000:
        print("Done! VDMA signaled completion.")
        break
    time.sleep(0.2)

out_buffer.invalidate()
unique_vals = np.unique(np.array(out_buffer))
print(f"Unique pixel values in output: {unique_vals[:10]}")

if len(unique_vals) > 1:
    print("\nSUCCESS: The AXI bus is un-jammed and data is flowing!")
else:
    print("\nFAILURE: Output buffer is still empty.")

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Original")
plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(np.array(out_buffer), cv2.COLOR_RGBA2RGB))
plt.title("Loopback Output")
plt.show()

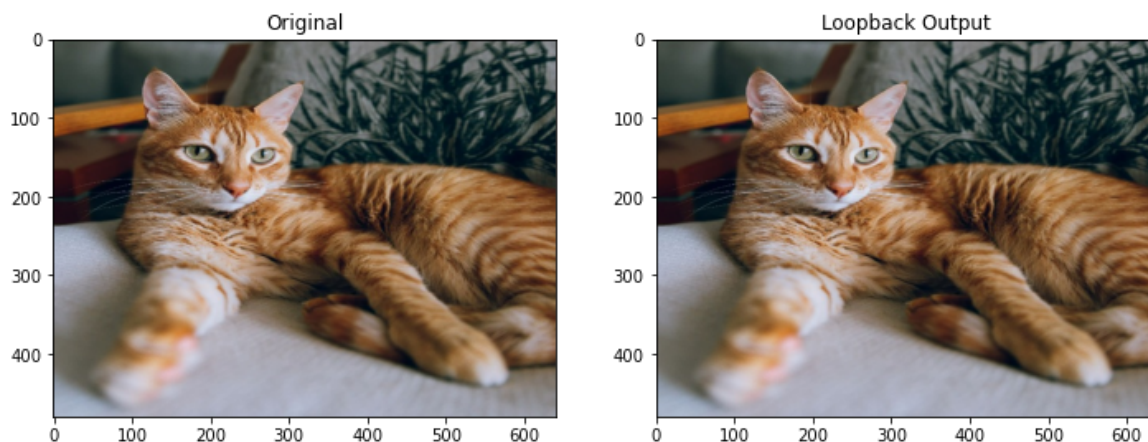
in_buffer.freebuffer()
out_buffer.freebuffer()
```



```
Fixing filenames...
Current files in folder:
['loop.hwh', 'loop.bit']
```

```
Loading Fresh Bitstream...
Loaded IPs: ['axi_vdma_0', 'img_pro_top_loop_bac_0', 'processing_system7_0']
Loading test.jpg...
Allocating memory...
Initializing VDMA...
Firing Transfer...
Polling VDMA Status...
  S2MM:0x14100 MM2S:0x10000
  S2MM:0x15100 MM2S:0x11000
Done! VDMA signaled completion.
Unique pixel values in output: [0 1 2 3 4 5 6 7 8 9]
```

SUCCESS: The AXI bus is un-jammed and data is flowing!



In [1]:

```
-----
-----
KeyboardInterrupt                                Traceback (most recent call
last)
Input In [1], in <cell line: 3>()
      1 from pynq import Overlay, MMIO, allocate
      2 import numpy as np
----> 3 import cv2
      4 import matplotlib.pyplot as plt
      5 import time

KeyboardInterrupt:
```

```

In [2]: from pynq import Overlay, MMIO, allocate
import numpy as np
import cv2
import matplotlib.pyplot as plt
import time
import os

# --- 1. SETTINGS ---
folder = "/home/xilinx/jupyter_notebooks/image_enhancer/Bitstreamfiles"
bit_name = "gray_test.bit" # <--- Change this to your actual filename
img_path = "/home/xilinx/jupyter_notebooks/image_enhancer/test.jpg"

IMG_WIDTH = 640
IMG_HEIGHT = 480
STRIDE = IMG_WIDTH * 4 # 4 bytes per pixel (RGBA)

# --- 2. LOAD HARDWARE ---
print("Loading Grayscale Bitstream...")
overlay = Overlay(folder + bit_name)
print("IPs Found:", list(overlay.ip_dict.keys()))

# --- 3. PREPARE IMAGE ---
print("Loading and resizing image...")
img = cv2.imread(img_path)
if img is None:
    raise ValueError(f"Could not find image at {img_path}")

img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT))
img_rgba = cv2.cvtColor(img, cv2.COLOR_BGR2RGBA)

# --- 4. ALLOCATE MEMORY ---
# Note: Even though it is grayscale, we usually keep 4 bytes (RGBA)
# to keep the VDMA settings simple.
in_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)
out_buffer = allocate(shape=(IMG_HEIGHT, IMG_WIDTH, 4), dtype=np.uint8)

in_buffer[:] = img_rgba
out_buffer[:] = 0

in_buffer.flush()
out_buffer.flush()

# --- 5. INITIALIZE VDMA ---
vdma_base = overlay.ip_dict['axi_vdma_0']['phys_addr']
vdma = MMIO(vdma_base, 0x10000)

print("Starting VDMA...")
# Reset MM2S and S2MM
vdma.write(0x00, 0x4)
vdma.write(0x30, 0x4)
time.sleep(0.1)

# Start both channels
vdma.write(0x00, 0x01) # MM2S Control
vdma.write(0x30, 0x01) # S2MM Control

```

```
# Set Physical Addresses
vdma.write(0x5C, in_buffer.physical_address) # MM2S Start Address 1
vdma.write(0xAC, out_buffer.physical_address) # S2MM Start Address 1

# Set HSIZE and STRIDE
vdma.write(0x54, STRIDE) # MM2S HSIZE
vdma.write(0x58, STRIDE) # MM2S STRIDE
vdma.write(0xA4, STRIDE) # S2MM HSIZE
vdma.write(0xA8, STRIDE) # S2MM STRIDE

# Fire the transfer by setting VSIZE
print("Firing Transfer...")
vdma.write(0xA0, IMG_HEIGHT) # S2MM VSIZE (Receiver first)
vdma.write(0x50, IMG_HEIGHT) # MM2S VSIZE (Transmitter second)

# --- 6. WAIT FOR COMPLETION ---
print("Polling status...")
for i in range(20):
    s2mm_status = vdma.read(0x34)
    mm2s_status = vdma.read(0x04)

    # Check for Error Bit (Bit 12) or Done/Idle bits
    if s2mm_status & 0x1000:
        print(f"Transfer Done/Error bit detected: {hex(s2mm_status)}")
        break
    time.sleep(0.1)

# --- 7. REFRESH AND SHOW DATA ---
out_buffer.invalidate()
result_img = np.array(out_buffer)

# Check if data actually changed
unique_vals = np.unique(result_img)
print(f"Unique pixel values in output: {len(unique_vals)}")

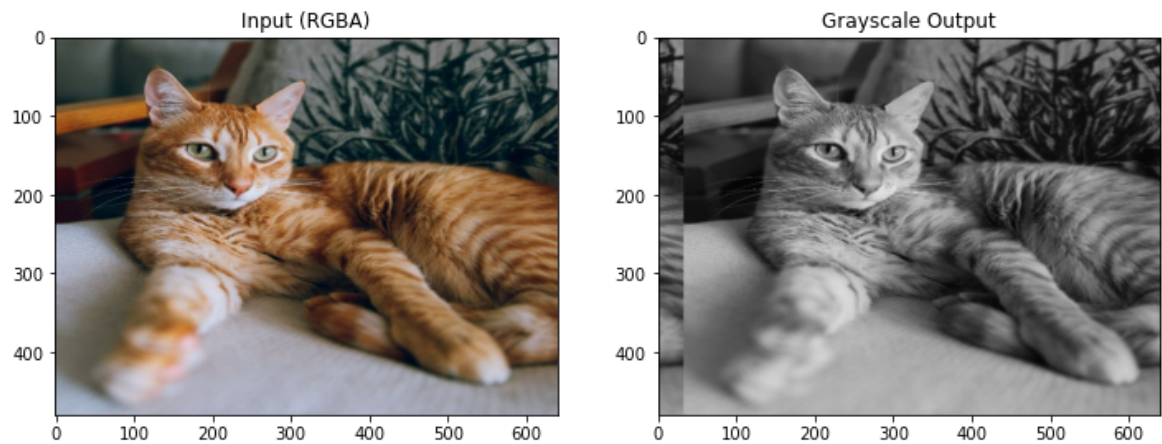
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2RGB))
plt.title("Input (RGBA)")

plt.subplot(1, 2, 2)
# We show as RGB, if grayscale is working, R=G=B and it will look gray
plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_RGBA2RGB))
plt.title("Grayscale Output")
plt.show()

# --- 8. CLEANUP ---
in_buffer.freebuffer()
out_buffer.freebuffer()
```

Loading Grayscale Bitstream...

```
IPs Found: ['axi_vdma_0', 'img_pro_top_loop_bac_0', 'processing_syste  
m7_0']  
Loading and resizing image...  
Starting VDMA...  
Firing Transfer...  
Polling status...  
Transfer Done/Error bit detected: 0x11000  
Unique pixel values in output: 238
```



In []: